

13) Security, Auth and Access

Last updated by | Mennell, Jacob | 19 May 2026 at 15:43 BST

Security, Auth, And Access

Purpose

This page covers the externally shareable parts of the solution's security posture, including authentication, role-based access control, and service boundary considerations.

Scope

The main security controls are applied at the API boundary. Authentication is token-based, role checks are explicit, and configuration supports controlled development-time exceptions.

Authentication Model

The API security code validates bearer tokens using Azure Active Directory-compatible JWT verification. The validation flow includes:

- resolving signing keys from a JWKS endpoint
- validating signature, issuer, audience, expiry, and related token claims
- attaching user identity information to request state for downstream use and logging

This means the API relies on an external identity provider rather than managing its own username and password model.

Authorisation Model

Authorisation is role-based. Route groups are protected by required role sets, and the security layer checks whether the user's token claims intersect with the required roles for the endpoint.

This provides a clear access model for:

- standard application users
- administrators
- system-level service actors

Development And Test Exceptions

The configuration includes toggles that can disable authentication or elevate users to admin roles for development or test scenarios. These are useful for local development, but they also reinforce an important documentation boundary: such switches are implementation conveniences, not part of the intended production security posture.

Access Boundaries In The Application

The repository suggests several layers of access control:

- API role enforcement at route level
- model-based user assignment and access records
- admin-only frontend routes for privileged functions

Together, these indicate that access is not only authentication-based but also constrained by business and application context.

Service Boundary Considerations

The browser UI does not appear to bypass the API for direct worker or database access. This is an important architectural security property because it keeps authentication, authorization, and audit-related checks concentrated at the API boundary.

Worker and data services operate behind that boundary and are reached through controlled backend paths.

What This Wiki Intentionally Does Not Include

For external sharing, this documentation intentionally excludes:

- secret values
- tenant-specific identifiers
- private endpoints or infrastructure addresses
- environment-specific credentials or connection details
- internal operational procedures for emergency access

Security Model Diagram



flowchart LR

```
User[Authenticated User] --> Token[Bearer Token]
Token --> API[API Security Layer]
API --> Roles[Role Validation]
Roles --> Routes[Protected Route Groups]
Routes --> Data[Claims, Documents, Feedback, Jobs]
```

Design Decisions

- Authentication is delegated to an external identity provider.
- Authorisation is explicit at route-registration level.
- Development-time bypasses exist but are configuration-controlled.
- Backend services are intentionally mediated through the API rather than exposed directly to the browser.